



INDIAN INSTITUTE OF SCIENCE, BANGALORE

COURSE PROJECT: CP274 FORMAL ANALYSIS AND CONTROL

Safe Learning for MADDPG with Control Barrier Certification for Long duration ground surveillance

Lokesh Bansal



Application: Multi Agent Systems



Figure 1: Swarm of drones for Surveillance



Figure 2: Autonomous Cars



Figure 3: Warehouse Robots

Proposed Environment:

Ground robots deployed for surveillance for long duration autonomy



Figure 4: Ground robots deployed for surveillance

Literature Survey

1. Control Barrier Functions: Theory and Applications

Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada

2. Robot ecology: Constraint-based control design for long duration autonomy

Magnus Egerstedt, Jonathan N. Pauli, Gennaro Notomista, Seth Hutchinson

3. Continuous control with deep reinforcement learning

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver & Daan Wierstra

4. Safe Exploration in Continuous Action Spaces

Gal Dalal, Krishnamurthy Dvijotham, Matej Vecerik, Todd Hester, Cosmin Paduraru, Yuval Tassa

5. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments

Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, Igor Mordatch



Key Concepts

1. Control Barrier Function 2nd Version
2. Policy Gradient Algorithms
3. Deep Deterministic Policy Gradient (DDPG) Algorithms
4. Safe Exploration in Continuous Action Spaces
5. Multi-Agent Deep Deterministic Policy Gradient (MADDPG)

Problem Formulation

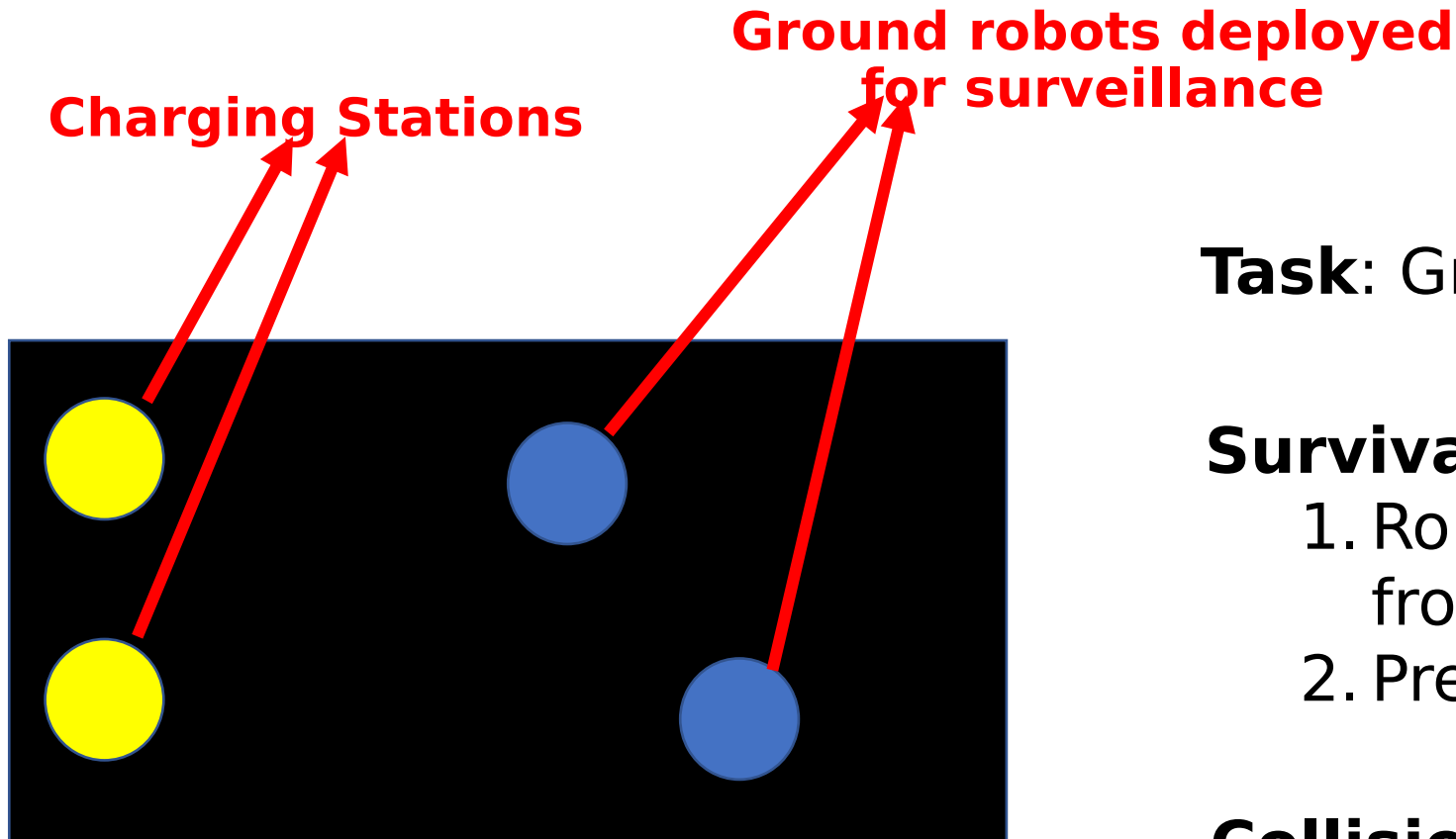


Figure 5: Long duration ground surveillance

Task: Ground Surveillance

Survivability constraints

1. Robot never gets stranded away from a charging station
2. Prevent overcharging

Collision avoidance

Notations and Equations[1,2]

- The dynamics of the augmented state X_i

$$\dot{x}_i = \begin{bmatrix} f(x_i) \\ \hat{f}(x_i) \end{bmatrix} + \begin{bmatrix} g(x_i) \\ \hat{g}(x_i) \end{bmatrix} u_i = F(x_i) + G(x_i)u_i.$$

- Dynamics of a robot is modelled by a control affine system

$$\dot{x}_i = f(x_i) + g(x_i)u_i,$$

- Energy dynamics are given by (the energy E_i stored in robot i's battery)

$$\dot{E}_i = \hat{f}(x_i) + \hat{g}(x_i)u_i$$

- Static mapping from robot i's state to its position p_i belongs to $\mathbb{R}^d$, $d = 2$ for ground robots or $d = 3$ for aerial robots

$$p : x_i \in \mathbb{R}^n \mapsto p_i \in \mathbb{R}^d$$

- Function that evaluates the energy that robot i requires to reach a charging station starting from position p_i

$$\rho_i : p_i \in \mathbb{R}^d \mapsto \rho_i(p_i) \in \mathbb{R}_{\geq 0}$$

K_d and k are energy constants, p_c is position of charging station associated with agent and d_{charge} is minimum distance required to get charged

$$\rho_i : \frac{K_d}{k} \log \frac{\|p_i - \hat{x}_i\|}{d_{charge}}$$

Barrier Certification for Safety[1,2]

For Survivability Constraints

- 1) By ensuring that each robot never gets stranded away from a charging station

$$h_{c,i}(\chi_i) = E_i - E_{min} - \rho_i(p(x_i)) \geq 0 \quad \forall i \in \{1, \dots, N\}$$

- 2) To prevent overcharging $h_{o,i}(\chi_i) = E_{max} - E_i \geq 0$.

By defining the logical and of these constraints, $h_{e,i}(\chi_i) = \min\{h_{c,i}(\chi_i), h_{o,i}(\chi_i)\}$
 $= h_{c,i} \wedge h_{o,i}$

For Collision Avoidance

$$h_s(\chi_i, \chi_j) = \|p(x_i) - p(x_j)\|^2 - \Delta^2 \geq 0$$

Combining the Barrier certificates for Collision avoidance and Survivability constraints:

$$h_i(\chi_i) = \min \left\{ \min_i \{h_{e,i}(\chi_i)\}, \min_{\substack{i,j \\ i \neq j}} \{h_s(\chi_i, \chi_j)\} \right\}$$

Barrier Certification for Surveillance[1,2]

- Consider N robots tasked with monitoring a compact and convex set Ω is subset of $\mathbb{R}^d$. We can define a measure of the coverage quality by defining a cost

$$J(x) = \sum_{i=1}^N \int_{\Omega_i} \|p(x_i) - q\|^2 \phi(q) dq.$$

Barrier function related to the task as $h_t(x)$

$=J(x)$

x is the ensemble state of the $\{\Omega_1, \dots, \Omega_N\}$ is the Voronoi tessellation of the set

The value $\phi(q) \in \mathbb{R}; \phi(q) \geq 0 \forall q \in \Omega$, encodes the importance of the point q

Where the quality of the sensor coverage associated with the point q decreases quadratically with the distance

$$\|p(x_i) - q\|$$

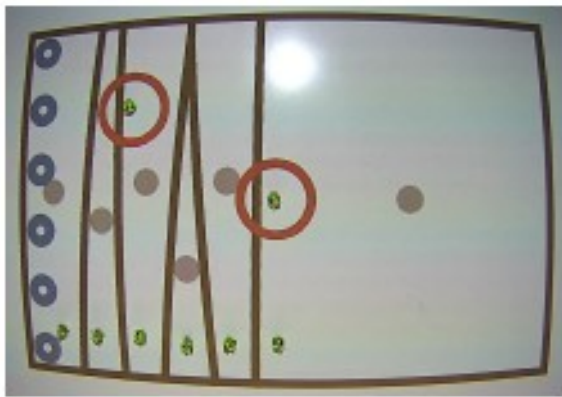
Given Approach in Literature[1,2]

$$\min_{u_1, \dots, u_N, \delta} \sum_{i=1}^N \|u_i\|^2 + \kappa|\delta|^2$$

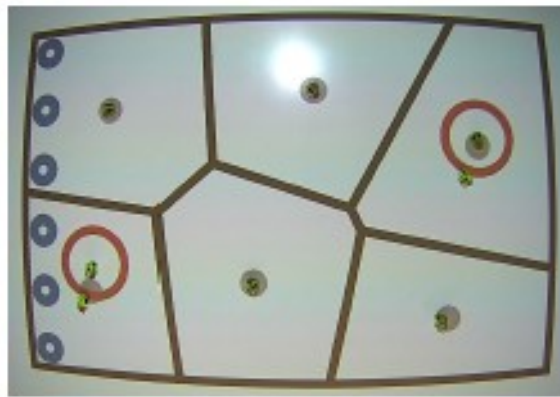
Each robot executes the input u_i solution of the following QP:

$$\begin{aligned} \text{s.t. } & L_F h_i(\chi_i) + L_G h_i(\chi_i) u_i \geq -\alpha(h_i(\chi_i)), \quad \forall i \\ & L_F h_t(\chi) + L_G h_t(\chi) u \geq -\alpha(h_t(\chi)) - \delta \end{aligned}$$

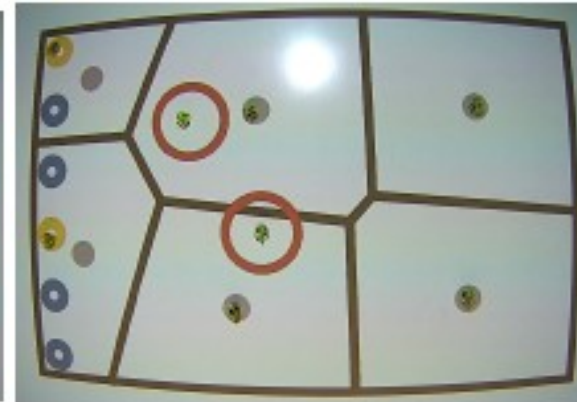
δ Relaxation variable (allows us to trade the execution of the coverage task for safety and energy, i. e., survivability)



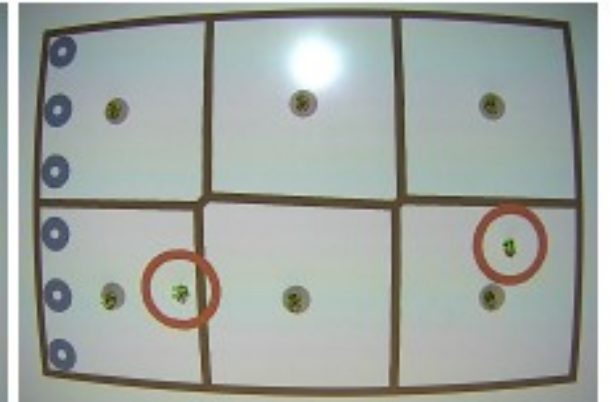
(a)



(b)



(c)



(d)

Proposed Approach using MADDPG with Barrier Certification[1,2]

Safety using Barrier Certification

$$\min_{u_1, \dots, u_N}, \quad \sum_{i=1}^N \|u_i\|^2$$
$$\text{s.t. } L_F h_i(\chi_i) + L_G h_i(\chi_i) u_i \geq -\alpha(h_i(\chi_i)), \quad \forall i$$

Surveillance using MADDPG

$$J(x) = \sum_{i=1}^N \int_{\Omega_i} \|p(x_i) - q\|^2 \phi(q) dq,$$

A safe control input $u_{i \text{ safe}}$ can be determined by solving the following Quadratic program

$$u_{\text{safe}}^i = \operatorname{argmin}_u \frac{1}{2} \|u - \hat{u}^i\|$$

$$\text{subject to } L_F h_i(\chi_i) + L_G h_i(\chi_i) u + \alpha(h_i(\chi_i)) \geq 0$$

DDPG Explained[3]

- DDPG is an **off-policy** algorithm
- DDPG can only be used for environments with **continuous action spaces**
- DDPG can be thought of as being deep Q-learning for continuous action spaces.
- It learns a Q-function and a policy: **Actor-Critic** Method.
- Uses off-policy data and the Bellman equation to learn the Q-function
- Uses the Q-function to learn the policy
- Computing the maximum over actions is a challenge in continuous action spaces
- DDPG deals with this by using a target policy network to compute an action which approximately maximizes $Q_{\phi_{\text{targ}}}$

DDPG Explained- Implementation[3]

DDPG uses four neural networks: a Q network, a deterministic policy network, a target Q network, and a target policy network

Parameters:

θ^Q : Q network

θ^μ : Deterministic policy function

$\theta^{Q'}$: target Q network

$\theta^{\mu'}$: target policy network

Key Equations

The Q-Learning Side of DDPG

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$$

$$Loss = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$$

DDPG Explained- Implementation [3]

Key Equations

The Policy Learning Side of DDPG

- Want to learn a deterministic policy $\mu_\theta(s)$ which gives the action that maximizes $Q_\phi(s, a)$
- Perform gradient ascent (with respect to policy parameter only) to solve
$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [Q_\phi(s, \mu_\theta(s))]$$

$$J(\theta) = \mathbb{E}[Q(s, a)|_{s=s_t, a_t=\mu(s_t)}]$$

$$\nabla_{\theta^\mu} J(\theta) \approx \nabla_a Q(s, a) \nabla_{\theta^\mu} \mu(s|\theta^\mu)$$

Exploration vs. Exploitation

To explore better, we add noise to their actions at training time

$$\mu'(s_t) = \mu(s_t|\theta_t^\mu) + \mathcal{N}$$

Target Network Updates

Target network parameters slowly track the learned networks via “soft updates,”

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

where $\tau \ll 1$

MADDPG with Control Barrier Certification Algorithm[5]

Initialize

Initialize replay buffer $\mathcal{D}$

For episode = 1 to m **do**:

Reset environment

Initialize state s

For time step $t = 1$ to max-episode-length **do**

Initialize a random process $\mathcal{N}_t$ for action exploration

For agent $i = 1$ to n **do**

Select action $a_t^i = \pi_{\theta_i}^i(s_t^i) + \mathcal{N}_t$

w.r.t the current policy and exploration

Obtain safety action $a_{t,safe}^i$ by solving the QP

based on the nominal action a_t^i

Update action $a_t^i \leftarrow a_{t,safe}^i$

End For

MADDPG with Control Barrier Certification Algorithm[5]

Execute actions $\{a_t^i\}_{i \in N}$

Observe reward r_t and new state s'

Store $(s, \{a_t^i\}_{i \in N}, r_t, s')$ in replay buffer $\mathcal{D}$

Update state $s \leftarrow s'$

For agent $i = 1$ to n do:

Sample a random minibatch of k samples (s_j, a_j, r_j, s'_j) from $\mathcal{D}$

Calculate the target state-action value:

$$y_j = r_j + \gamma Q_{\pi'}^i(s'_j, a'^1, \dots, a'^n) |_{a'^i = \pi'^i(s'_j)}$$

Update critic by minimizing the loss:

$$L(\theta_i) = \frac{1}{k} \sum_j (y_j - Q_{\pi}^i(s_j, a_j^1, \dots, a_j^n))^2$$

Update actor using the gradient of loss:

$$\nabla_{\theta_i} J \approx \frac{1}{k} \sum_j \nabla_{\theta_i} \pi^i(s_j^i) \nabla_{\theta_i} Q_{\pi}^i(s_j, a_j^1, \dots, a_j^n) |_{a^i = \pi^i(s_j^i)}$$

Update target network parameters:

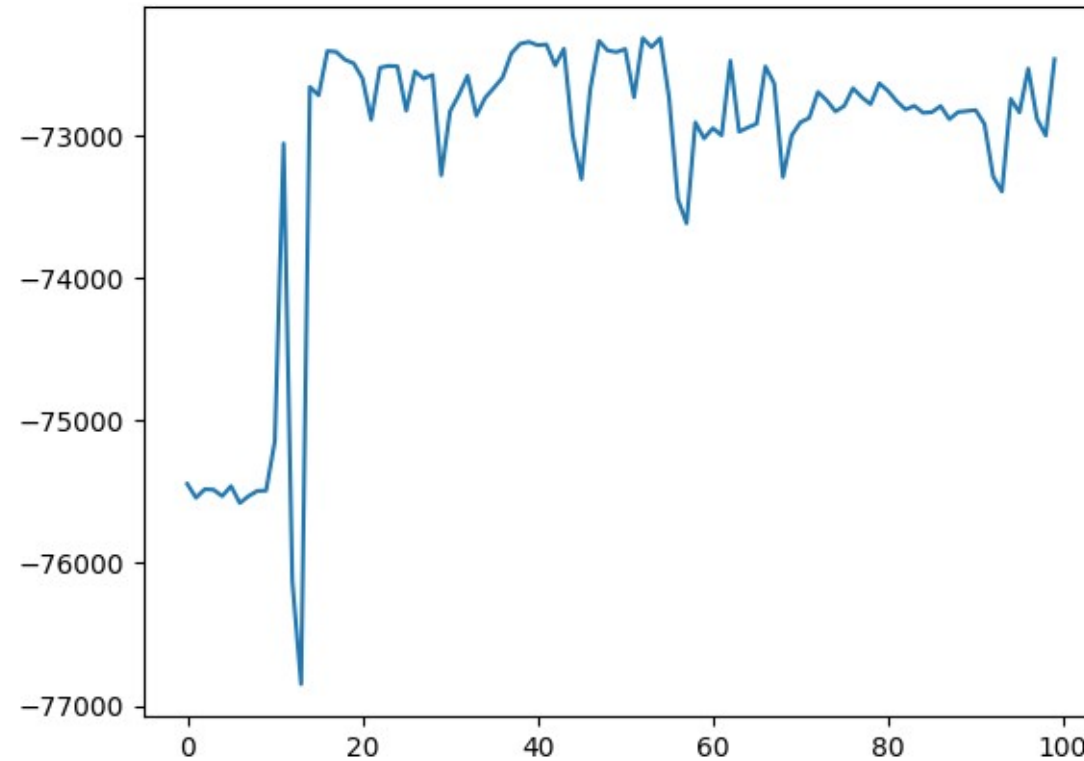
$$\theta'_i \leftarrow \xi \theta_i + (1 - \xi) \theta'_i$$

End For

End For

End For

Training



1. Number of episodes: 10,000
2. Time steps per episode (episode length): 100
3. Save rate: after every 100 episodes

Safe Learning for MADDPG with CBF for Long duration ground surveillance

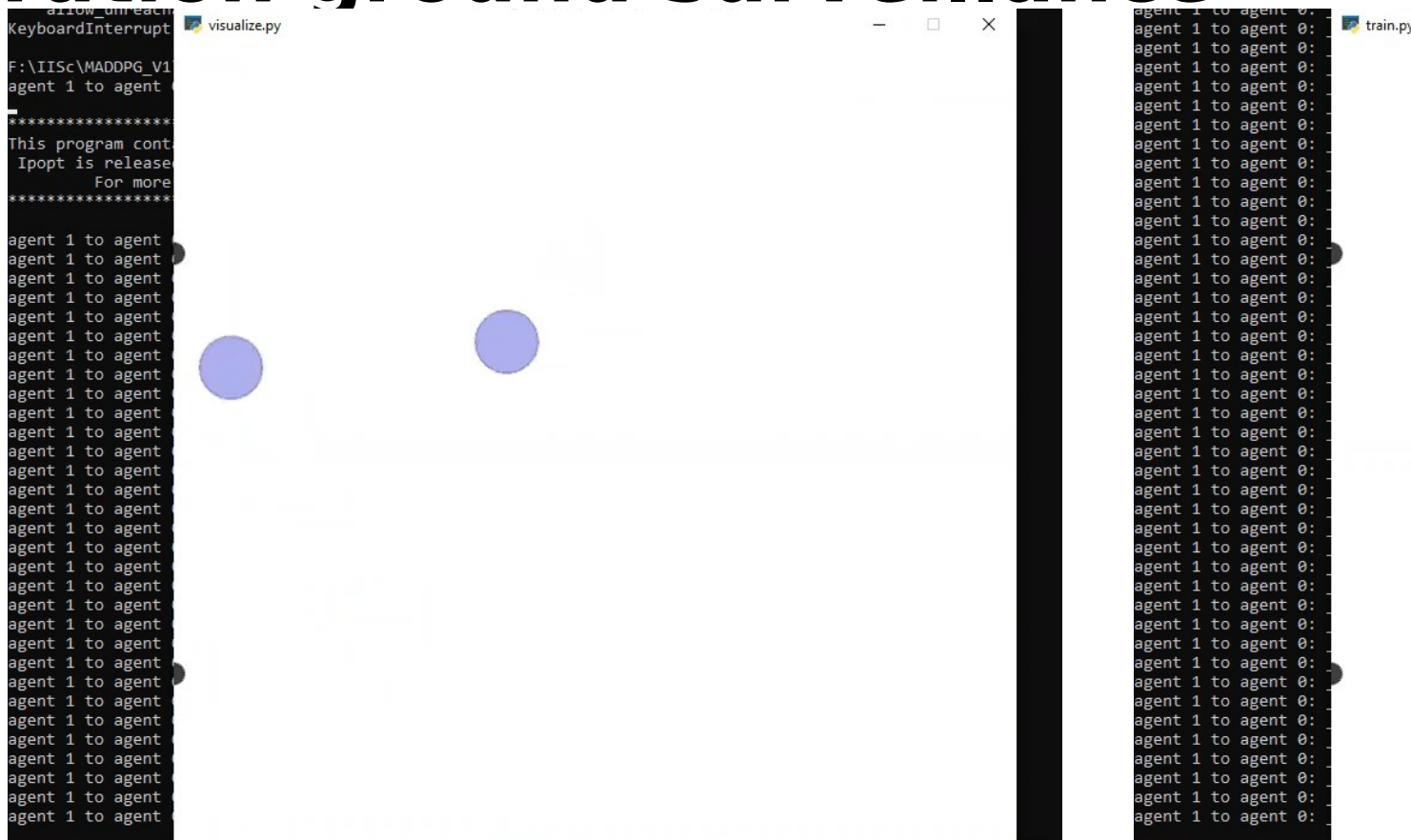


Figure 6: Before training



Figure 7: After training

1. Task is to deploy battery driven autonomous agents for surveillance.
2. Train the agents for the autonomous operation with safety constraints (Battery must not drained out).
3. For training the agent multi agent deep deterministic policy gradient algorithm is used with Control Barrier Function (CBF) for safety constraint.

Thank You

